# CPE223 Computer Architecture
## PS7 Processor Design

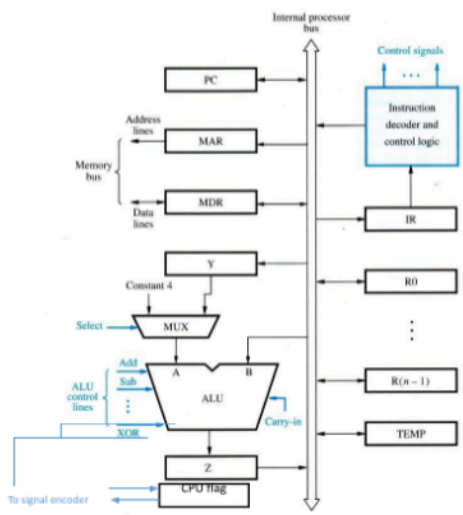
**Group name: ComAhhh!**

**Member:**

| | | | |
|---|---|---|---|
| **1.** | KITTITHAT | DISTHANAKORNKUN | 67070501004 |
| **2.** | JETANIN | NAITHO | 67070501011 |
| **3.** | NAPHAT | UTABUAWONG | 67070501015 |
| **4.** | THANATIP | NITINANTAKUL | 67070501023 |
| **5.** | NANTAKORN | PINSUPAPORN | 67070501028 |
| **6.** | KAWINPOP | CHURARI | 67070501079 |

**Introduction:**

Instruction set can be divided into four main categories including arithmetic/logic operation, transfer data, fetch/store data, and branch with several addressing modes to access the memory. In this lab, we are going to figure out the stages and data paths of each instructions and find out how to design a hardwire control for the processor to perform some selected instructions.




| Instruction | |
|---|---|
| ADD | Rn,off(Rn) |
| ADD | Rn,#value |
| ADD | Rn,Rn |
| MOV | Rn,(Rn) |
| MOV | (index),Rn |
| BZ | (Rn) |
| BZ | +#relative |
| B | #Absolute |

*Figure 1 Single bus processor*

For our lab, we will use single bus system as illustrated in figure 1. Then we will use only a few of them to grasp the idea on how to design the stages and data paths. The instruction set is listed here consists of add, move, and branch operations.

**Task 1:** Design stages and data paths of each instruction to get the idea on the effect of addressing mode and complex instruction on the stages and data paths. There are eight instructions to divide among your group members to come up with stages and data paths.

Ex. Add (R3), R1

| Step | Action |
|---|---|
| 1 | $PC_{out}$ , $MAR_{in}$ , Read, Select4,Add, $Z_{in}$ |
| 2 | $Z_{out}$ , $PC_{in}$ , WMFC |
| 3 | $MDR_{out}$ , $IR_{in}$ |
| 4 | $R3_{out}$ , $MAR_{in}$ , Read |
| 5 | $R1_{out}$ , $Y_{in}$ , WMFC |
| 6 | $MDR_{out}$ , SelectY, Add, $Z_{in}$ |
| 7 | $Z_{out}$ , $R1_{in}$ , End |

**Instruction:**

1. **ADD Rn, off(Rn)**

| Step | Action |
|---|---|
| 1 | $PC_{out}$ , $MAR_{in}$ , Read , Select 4 , Add , $Z_{in}$ |
| 2 | $Z_{out}$ , $PC_{in}$ , WMFC |
| 3 | $MDR_{out}$ , $IR_{in}$ |
| 4 | $PC_{out}$ , $MAR_{in}$ , Read |
| 5 | $Rn_{out}$ , $Y_{in}$ , WMFC |
| 6 | $MDR_{out}$ , Select Y , Add , $Z_{in}$ |
| 7 | $Z_{out}$ , $MAR_{in}$ , Read |
| 8 | WMFC |
| 9 | $MDR_{out}$ , Select Y , Add , $Z_{in}$ |
| 10 | $Z_{out}$ , $Rn_{in}$ , End |

2. **ADD Rn, #value**

| Step | Action |
|---|---|
| 1 | $PC_{out}$ , $MAR_{in}$ , Read , Select 4 , Add , $Z_{in}$ |
| 2 | $Z_{out}$ , $PC_{in}$ , WMFC |
| 3 | $MDR_{out}$ , $IR_{in}$ |
| 4 | $PC_{out}$ , $MAR_{in}$ , Read |
| 5 | $Rn_{out}$ , $Y_{in}$ , WMFC |
| 6 | $MDR_{out}$ , Select Y , Add , $Z_{in}$ |
| 7 | $Z_{out}$ , $Rn_{in}$ , End |

## 3. ADD Rn, Rn

| Step | Action |
|---|---|
| 1 | $PC_{out}$ , $MAR_{in}$ , Read , Select 4 , Add , $Z_{in}$ |
| 2 | $Z_{out}$ , $PC_{in}$ , WMFC |
| 3 | $MDR_{out}$ , $IR_{in}$ |
| 4 | $Rn_{out}$ , Select Y , Add , $Z_{in}$ |
| 5 | $Z_{out}$ , $Rn_{in}$ , End |

## 4. MOV (Rn), Rn

| Step | Action |
|---|---|
| 1 | $PC_{out}$ , $MAR_{in}$ , Read , Select 4 , Add , $Z_{in}$ |
| 2 | $Z_{out}$ , $PC_{in}$ , WMFC |
| 3 | $MDR_{out}$ , $IR_{in}$ |
| 4 | $Rn_{out}$ , $MAR_{in}$ |
| 5 | $Rn_{out}$ , $MDR_{in}$ , Write |
| 6 | WMFC , End |

## 5. MOV (Index), Rn

| Step | Action |
|---|---|
| 1 | $PC_{out}$ , $MAR_{in}$ , Read , Select 4 , Add , $Z_{in}$ |
| 2 | $Z_{out}$ , $PC_{in}$ , WMFC |
| 3 | $MDR_{out}$ , $IR_{in}$ |
| 4 | $PC_{out}$ , $MAR_{in}$ , Read |
| 5 | WMFC |
| 6 | $MDR_{out}$ , $MAR_{in}$ |
| 7 | $Rn_{out}$ , $MDR_{in}$ , Write |
| 8 | WMFC , End |

## 6. BZ (Rn)

| Step | Action |
|---|---|
| 1 | $PC_{out}$ , $MAR_{in}$ , Read , Select 4 , Add , $Z_{in}$ |
| 2 | $Z_{out}$ , $PC_{in}$ , WMFC |
| 3 | $MDR_{out}$ , $IR_{in}$ |
| 4 | If Z=0 , End |
| 5 | $Rn_{out}$ , $MAR_{in}$ , Read |
| 6 | WMFC |
| 7 | $MDR_{out}$ , $PC_{in}$ , End |

## 7. BZ +#relative

| Step | Action |
|---|---|
| 1 | $PC_{out}$ , $MAR_{in}$ , Read , Select 4 , Add , $Z_{in}$ |
| 2 | $Z_{out}$ , $PC_{in}$ , WMFC |
| 3 | $MDR_{out}$ , $IR_{in}$ |
| 4 | If Z=0 , End |
| 5 | $PC_{out}$ , $MAR_{in}$ , Read |
| 6 | WMFC |
| 7 | $MDR_{out}$ , $Y_{in}$ |
| 8 | $PC_{out}$ , Select Y , Add , $Z_{in}$ |
| 9 | $Z_{out}$ , $PC_{in}$ , End |

## 8. B #Absolute

| Step | Action |
|---|---|
| 1 | $PC_{out}$ , $MAR_{in}$ , Read , Select 4 , Add , $Z_{in}$ |
| 2 | $Z_{out}$ , $PC_{in}$ , WMFC |
| 3 | $MDR_{out}$ , $IR_{in}$ |
| 4 | $PC_{out}$ , $MAR_{in}$ , Read |
| 5 | WMFC |
| 6 | $MDR_{out}$ , $PC_{in}$ , End |

**Task 2:** Design a hardwire circuit to generate control signals for $Y_{in}$, $Z_{in}$, $Rn_{in}$, and MARin from the selected instruction from the previous task. The instructions that you have to consider are three ADDs, and BZ (Rn).

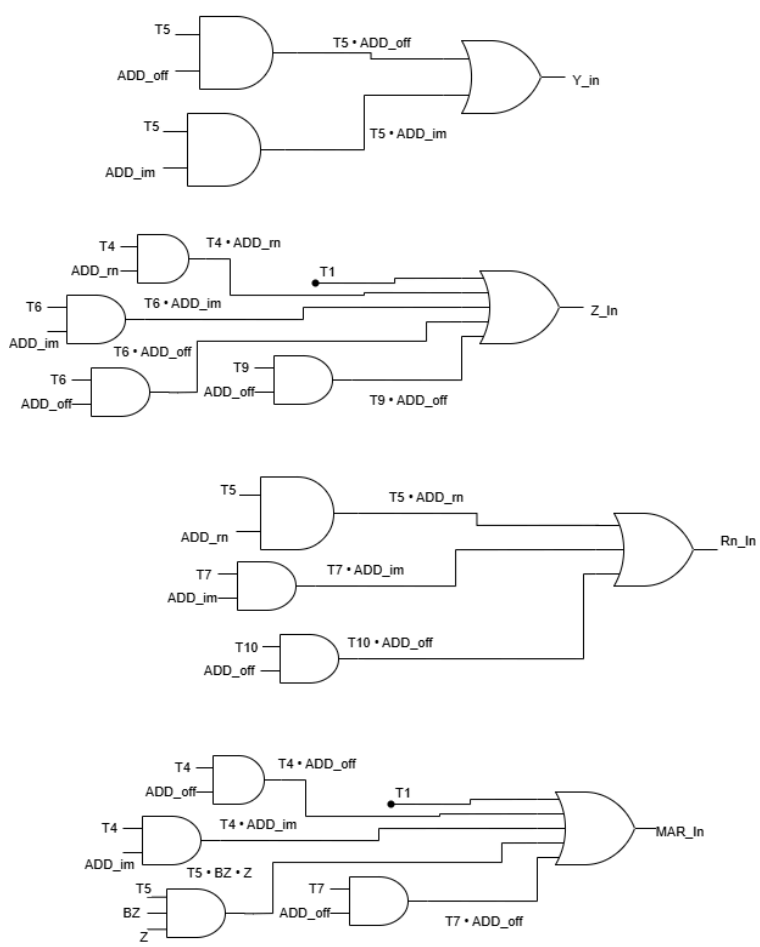
**Instruction:**

**ADD Rn , off(Rn)**
**ADD Rn , #value**
**ADD Rn , Rn**
**BZ (Rn)**

**Boolean Function:**

- $\mathbf{Y_{in}}$ = $(T_5 \cdot ADD_off) + (T_5 \cdot ADD_im)$
- $\mathbf{Z_{in}}$ = $T_1 + (T_4 \cdot ADD_rn) + (T_6 \cdot ADD_im) + (T_6 \cdot ADD_off) + (T_9 \cdot ADD_off)$
- $\mathbf{Rn_{in}}$ = $(T_5 \cdot ADD_rn) + (T_7 \cdot ADD_im) + (T_{10} \cdot ADD_off)$
- $\mathbf{MAR_{in}}$ = $T_1 + (T_4 \cdot ADD_off) + (T_4 \cdot ADD_im) + (T_5 \cdot Bz \cdot Z) + (T_7 \cdot ADD_off)$

**Circuit Diagram:**



**Discussion:**

วงจรควบคุมที่ออกแบบในงานนี้เป็นวงจรแบบ hardwired control ซึ่งทำหน้าที่สร้างสัญญาณควบคุมให้กับส่วนต่าง ๆ ของ datapath ได้แก่ (Y_{in}), (Z_{in}), (Rn_{in}) และ (MAR_{in}) โดยสัญญาณเหล่านี้ถูกสร้างจากการผสมกันของ สัญญาณเวลา (Timing signals) เช่น (T1, T4, T5, T6, T7, T9, T10) และ สัญญาณบอกชนิดคำสั่ง (Instruction signals) ได้แก่ ADD_off, ADD_im, ADD_rn และ BZ ตาม Boolean function ที่กำหนดไว้ แต่ละสัญญาณควบคุมจะเกิดขึ้นเฉพาะในช่วงเวลาและเงื่อนไขของคำสั่งที่เกี่ยวข้องเท่านั้น เช่น การเปิดใช้งานรีจิสเตอร์หรือการส่งข้อมูลเข้าสู่รีจิสเตอร์ต่าง ๆ ภายใน datapath ของหน่วยประมวลผล

การสร้างวงจรทำโดยใช้ AND gate เพื่อรวมเงื่อนไขของสัญญาณเวลาและชนิดคำสั่งให้เป็น product terms จากนั้นนำผลลัพธ์ของแต่ละเงื่อนไขมารวมกันด้วย OR gate เพื่อสร้างสัญญาณควบคุมสุดท้ายของแต่ละโมดูล วิธีการนี้ช่วยให้สามารถกำหนดพฤติกรรมของระบบได้อย่างชัดเจนตามลำดับขั้นของการทำงานของคำสั่งในแต่ละ clock cycle นอกจากนี้ การแยกวงจรออกเป็นสัญญาณควบคุมแต่ละตัว เช่น (Y_{in}), (Z_{in}), (Rn_{in}) และ (MAR_{in}) ยังช่วยให้โครงสร้างของวงจรเข้าใจได้ง่ายและสามารถตรวจสอบความถูกต้องของเงื่อนไขการทำงานได้สะดวก ซึ่งเป็นแนวทางพื้นฐานของการออกแบบ control unit แบบ hardwired ในสถาปัตยกรรมคอมพิวเตอร์